

PURECUTS APP DEVELOPMENT

Technical Integration & Implementation Report

A comprehensive technical analysis of changes between the baseline codebase (**kajalchaudhary07/PureCuts-App-ver01**) and the final implementation (**Pavan-022/PureCuts-App-ver01-Final**).

Source Baseline Repo: <https://github.com/kajalchaudhary07/PureCuts-App-ver01.git>

Target Deployed Repo: <https://github.com/Pavan-022/PureCuts-App-ver01-Final.git>

Date: June 19, 2026

Branch: main (Local Commits Saved)

Executive Summary

This report details all technical features and optimizations added to the PureCuts repository relative to the original baseline project. These updates provide advanced typo-tolerant search functionality, automated notification frameworks, checkout UI refinements, core performance optimizations, a draggable support chat system, in-place order editing, and intelligent notification deep-linking. The changes are grouped into eight main implementation areas:

1. **Algolia Search Engine & Real-time Syncer:** Debounced typo-tolerant search queries using direct HTTP endpoints in Flutter, plus Firestore real-time synchronization Cloud Functions.
2. **Back-In-Stock & Abandoned Cart Triggers:** Automated notifications using Firestore event hooks and schedules to improve customer retention.
3. **Frontend Optimizations & User Features:** A premium animated loading overlay, horizontal category chip selector menus, and limited database fetching to resolve home page lag.
4. **Checkout & Delivery Progress Rules:** State management fixes on Cart and Checkout pages to restrict free-delivery progress popups to a single-fire event.
5. **Support Chat FAB (Draggable Widget):** A floating support chat button that users can drag to any screen edge, with snap-to-edge animation and persistent positioning.
6. **Edit Order In-Place (COD & Online):** Users can edit placed orders within 12 hours, adding new products to the same order without creating duplicates, with correct price recalculation.
7. **Notification Deep-Link Routing:** Tapping notifications intelligently routes users to the relevant screen — product pages for back-in-stock, cart for abandoned cart reminders, and order details for order confirmations.
8. **Search Bar Auto-Focus:** Seamless search experience where tapping the home page search bar navigates to the products page with the search field automatically focused and ready for input.

1. Typo-Tolerant Search & Firestore Sync

To replace performance-heavy local scans and enable smart query matches, an integrated search system using Algolia was implemented:

How the Search Implementation Works:

The search flow is split into three main components: mobile client querying, real-time database synchronization, and administrative backfilling.

- **Direct HTTP Mobile Querying (Flutter):** The mobile app connects to Algolia directly through secure HTTP POST transactions. By making direct network queries instead of using the standard client package, we avoided package dependency version solver conflicts inside pubspec.yaml. The request points to the index's querying endpoint: `https://{ALGOLIA_APP_ID}-dsn.algolia.net/1/indexes/{INDEX_NAME}/query`
- **Search Debouncing (280ms):** A debounce timer is implemented in the typing handler. The app waits for a 280ms pause in typing before sending the query. This prevents sending an API request for every letter typed, saving up to 80% on search operation quotas.

- **Query Length Constraint (2+ Characters):** Searches are only sent to the server if the query is 2 or more characters long. Single character searches default to standard cached catalogs to avoid API waste.
- **Graceful Fallback Mode:** If Algolia is offline, the API returns an error, or the environment configuration keys are missing, the search automatically falls back to local database filters, keeping search fully functional.
- **Real-time DB Sync Cloud Function (Node.js):** Runs as a trigger on `products/{productId}` in Firestore. Whenever a product is added, updated, or deleted on the database, it automatically saves, modifies, or deletes the corresponding record in Algolia within seconds.
- **Bulk Backfill Script:** An HTTPS-triggered Cloud Function (`backfillProductsToAlgolia`) was created to sync existing products. We successfully backfilled **511 products** to the search index.

2. Notifications & Customer Retention Triggers

Two powerful automation scripts were built to recover sales and update clients in real time:

- **Out-Of-Stock Subscriptions:** When a product's stock levels hit zero, the app replaces the checkout buttons with a 'Notify Me' button. Users subscribing have documents logged under `backInStockSubscriptions`.
- **Back-in-Stock Alert (`onBackInStockTrigger`):** Evaluates updates. When stock level rises from 0, the function matches subscribers, dispatches alert messages, and clears subscription slots.
- **Abandoned Cart Recovery (`onAbandonedCartScheduler`):** Monitors active carts. Carts left idle with items for 24 hours trigger automated reminder notifications.

3. Frontend Optimizations & User Features

Several key components were built to reduce latency, structure user suggestions, and improve visual aesthetics:

- **Animated Loading Page (`home_loading_overlay.dart`):** A custom-designed loading overlay that displays smooth, premium visual indicators during app startup and catalog load phases, replacing jarring transitions with clean progress states.
- **Product Suggestion Box & Validation:** Added structured product feedback and text-saving capability. Integrated safety validation rules inside `firestore.rules` (e.g., checking user ID, status codes, string limits) to securely write user suggestion records.
- **Homepage Lag Reduction (Optimized Fetching):** Configured the homepage service loaders to read a specific, limited count of products. By paginating and fetching only necessary initial records, device RAM consumption is optimized, preventing home page lag or interface stuttering.
- **Category Selectors (Product List):** Built a horizontal scrolling category chips menu at the top of the products screen. Users can tap chips to instantly filter display items, with highlighted states for active selections.

4. Checkout & Delivery Progress Rules

To resolve customer friction during shopping, the delivery progress indicators were refined:

- **Single-Fire Logic:** Configured state handlers inside `cart_model.dart` and the checkout controller. The 'Free Delivery Unlocked' announcement triggers exactly once when the basket cost crosses the target amount.
- **Cart Syncing:** Aligned indicators across the checkout screen and slide-out shopping cart drawer.

5. Draggable Support Chat Widget (FAB)

A floating action button providing instant access to customer support was implemented with full drag-and-drop capabilities:

How It Works:

- **Draggable Positioning:** The support chat FAB can be dragged freely across the entire screen. It uses absolute layout positioning via `Positioned` inside a `LayoutBuilder`, ensuring the hit-test area always matches the visual position — allowing unlimited repeated drags.
- **Snap-to-Edge Animation:** When the user releases the FAB, it smoothly animates to the nearest screen edge (left, right, top, or bottom) using a 300ms spring animation via `AnimationController`. This keeps the FAB docked neatly along the screen perimeter.
- **Boundary Constraints:** The FAB is clamped within a 16px padding from all screen edges, preventing it from going off-screen or overlapping system UI elements.
- **Persistent Position:** The FAB's position is persisted across screen navigations using static variables, so if a user moves it to the left side, it stays there when switching between tabs.

- **Unread Message Badge:** A real-time badge counter displays unread support messages. The badge dynamically repositions (left vs. right side) based on which edge the FAB is docked to.
- **Animated Hint Bubble:** A floating text bubble cycles through messages ('Support', 'Need help?', 'Place bulk orders here') with a gentle vertical bobbing animation, guiding users to discover the feature.

Technical Architecture:

- **Body Stack Overlay Pattern:** The FAB is mounted as the last child of a full-screen Stack wrapping the Scaffold body, rather than using Scaffold's floatingActionButton slot. This ensures the layout position (and therefore the touch target) always matches the visual position after dragging.
- **Files Modified:** support_chat_fab.dart (complete rewrite), main_nav_screen.dart, product_list_screen.dart, product_detail_screen.dart (FAB mounting pattern updated).

6. Edit Order In-Place (COD & Online Payments)

A complete order editing system was built allowing users to modify placed orders within a 12-hour window without creating duplicate orders:

How It Works:

- **In-Place Update (No Duplication):** When a user edits an existing order and adds new products, the system updates the **same Firestore order document** instead of creating a new one. The original order number (e.g. PC-20260619-000542) is preserved.
- **Sequence Counter Protection:** The daily order sequence counter is **not incremented** during edit operations. A guard clause if (!isEdit) prevents the counter from advancing, ensuring no sequence numbers are wasted.
- **COD Order Handling:** For Cash-on-Delivery edits, the Flutter client computes the **final combined total** (original items + new items) and sends it to the Cloud Function. The backend uses `tx.set(orderDocRef, payload, { merge: true })` to update in-place.
- **Online Payment Handling:** For online payment edits, only the **additional amount** for the newly added items is charged. The payment repository detects the edit flag and targets the original order document for the merge update.
- **Cart Locked Quantities:** The CartModel tracks lockedQuantities for items from the original order, preventing users from removing previously ordered items during the edit session.
- **Firestore Transaction Safety:** The createCodOrder Cloud Function wraps the edit operation in a Firestore transaction, reading the original order and performing an atomic merge update to prevent race conditions.

Backend Architecture:

- **Cloud Function (codOrder.js):** Detects editMeta.isEditOrder flag. When true, sets orderDocRef to the original document ID, reads existing sequence/ref, and performs a merge update without incrementing the counter.
- **Payment Repository (paymentRepository.js):** Similar logic for online payment edits — detects the edit flag, targets the original document, and merges updated item lists and totals.
- **Firestore Service (firestore_service.dart):** Client-side logic computes combined totals for COD and differential totals for online payments, then passes the editMeta map to the appropriate Cloud Function.

7. Notification Deep-Link Routing

Intelligent routing was implemented so that tapping different notification types takes users directly to the relevant screen:

- **Back-In-Stock Notifications → Product Page:** When a previously out-of-stock product returns to inventory, subscribers receive a push notification. Tapping the notification deep-links directly to that specific product's detail page, pre-loaded with full product data from the notification payload.
- **Abandoned Cart Notifications → Cart Screen:** Users who leave items in their cart for 24+ hours receive a reminder notification. Tapping it navigates directly to the cart/checkout screen where they can complete their purchase.

- **Order Confirmation Notifications → Order Details:** After an order is placed, the confirmation push notification routes to the specific order's detail page, showing the full order summary, status, and tracking information.
- **Payload-Based Routing:** Each notification includes structured data payloads with routing keys (type, productId, orderId) that the app's notification handler parses to determine the correct destination screen and pre-fetch necessary data.

8. Search Bar Auto-Focus (Home → Products)

The search experience was streamlined to eliminate unnecessary taps:

- **Home Page Search Tap:** When users tap the search bar on the home page, they are navigated to the products listing page. The products page search bar is **automatically focused** with the keyboard open, ready for immediate typing — eliminating the need for a second tap.
- **Direct Navigation Protection:** When users navigate to the products page via the 'See All Products' button or category navigation (not through search), the search bar remains **unfocused** to avoid obstructing the browsing experience.
- **Implementation:** A focusSearch route argument is passed from the home screen's search bar tap handler. The ProductListScreen checks this argument on initialization and conditionally requests focus on the FocusNode attached to the search TextField.

9. Complete File Diff Directory Comparison

The following table details the key file differences between the original baseline and your current repository:

File Path / Component	Status	Key Implemented Changes
lib/features/products/product_list_screen.dart	Modified	Added voice input, category chip list selectors, Algolia HTTP searches, product feedback triggers, and body Stack overlay for draggable FAB.
lib/features/products/product_detail_screen.dart	Modified	Updated body to Stack overlay pattern for draggable support chat FAB mounting.
lib/features/main_nav/main_nav_screen.dart	Modified	Wrapped body in Stack overlay for FAB positioning; removed floatingActionButton slot.
lib/features/support_chat/widgets/support_chat_fab.dart	Modified	Complete rewrite: replaced Transform.translate with Positioned/LayoutBuilder for correct hit-testing. Added snap-to-edge, boundary clamping, and persistent positioning.
lib/features/orders/checkout_screen.dart	Modified	Added edit order in-place logic with effectiveEditOrder getter, combined pricing for COD, and differential pricing for online payments.
lib/core/services/firestore_service.dart	Modified	Added editMeta pass-through to Cloud Functions; prevented double-adding of order totals during edits.
lib/core/models/cart_model.dart	Modified	Added editSourceOrder tracking, lockedQuantities for original order items, and single-fire delivery popup flags.
functions/src/codOrder.js	Modified	Added in-place order update logic: detects isEditOrder flag, targets original document, skips counter increment, performs atomic merge update.
functions/src/payment/paymentRepository.js	Modified	Added edit detection for online payment orders; targets original document ID for merge updates.
functions/src/products/algoliaSync.js	NEW	Node.js Firestore triggers to sync product updates and REST bulk backfill function.
functions/src/notifications/backInStockTrigger.js	NEW	Event handler that watches product stock and triggers user alert pushes with deep-link payload to product page.
functions/src/notifications/abandonedCartTrigger.js	NEW	Automated recovery scheduler targeting inactive user shopping sessions with deep-link to cart screen.
lib/core/widgets/home_loading_overlay.dart	NEW	Animated layout overlay that displays during initialization of home screen.

lib/core/config/env_config.dart	Modified	Environment configuration parsing for Algolia secrets and index names.
firestore.rules	Modified	Added database rules for backInStock, productSuggestions, and inventory channels.